Proceedings of Machine Learning Research **318** (2026)
The 39th Canadian Conference on Artificial Intelligence

Canadian AI 2026
25–29 May 2026, Vancouver, BC

Execution-Aware A* Search for Cross-Exchange Stablecoin Arbitrage

Kevin Litvin
Simon Fraser University

Abstract

Cross-exchange cryptocurrency arbitrage enables low-risk profit from price discrepancies across exchanges, yet existing approaches employ negative cycle detection that targets opportunity identification rather than execution feasibility. We introduce an execution-aware pathfinding framework using A* search with domain-specific guidance heuristics, applied to stablecoins, a unique asset class exceeding $300 billion in market capitalization that bridges cryptocurrency and fiat currency, offering a novel dataset for arbitrage research. The problem is modelled as a weighted directed graph where nodes represent (exchange, stablecoin) pairs across 12 centralized exchanges and edges encode real-world costs including fees, slippage, gas, transfer delays, and exchange reliability. Three guidance heuristics and a multi-start strategy are evaluated over 7,200 search instances. Our slippage-aware heuristic h_2 reduces node expansions by 29% relative to Dijkstra while matching its profit, demonstrating that domain-specific heuristics can meaningfully improve execution feasibility in real-time arbitrage planning. Code: https://github.com/kevinl03/Stablecoin-CrossExchange-Arbitrage.

Keywords: stablecoin arbitrage, A* search, cross-exchange trading, execution-aware pathfinding, cryptocurrency, graph algorithms

1. Introduction

We formulate cross-exchange stablecoin arbitrage as a graph-planning problem and develop a route planner that computes executable trade-and-transfer paths using A* search with domain-specific heuristics and live market data via CCXT[1]. Our methodology addresses four research questions: (1) Do certain heuristics consistently outperform others on specific trading-instance classes? (2) How does each heuristic influence A*'s node-expansion behaviour? (3) How does performance change as structural difficulty increases? (4) When heuristics return suboptimal routes, how close are they to the best-known solution?

Stablecoins, with a combined market capitalization exceeding $300 billion[2], serve as the primary liquidity bridges in cryptocurrency markets. Because centralized exchanges operate independently, price discrepancies frequently arise from fragmented liquidity, withdrawal delays, and blockchain congestion. Unlike volatile cryptocurrencies, stablecoins operate within a bounded price band anchored to fiat currency, reframing arbitrage into a constrained execution-optimization problem. The growing institutional relevance of digital assets, underscored by the U.S. Strategic Bitcoin Reserve in 2025[3], further motivates research into this infrastructure.

The four key challenges addressed are: **Liquidity -** order-book depth limits deployable capital, **Slippage -** large orders incur nonlinear price impact, **Latency -** blockchain confirmations may exceed the arbitrage window, and **Reliability -** exchanges may suspend withdrawals or experience API instability. These constraints interact, and our heuristic-guided A* framework models these trade-offs to prioritize realistically executable routes.

2. Related Work

Most prior work models arbitrage as negative-weight cycle detection via Bellman–Ford[4–10], focusing on opportunity identification rather than execution feasibility. Cross-exchange settings introduce blockchain latency, withdrawal fees, and liquidity fragmentation[11, 12], while

risk-aware pathfinding[13, 14] and slippage models[15] remain disconnected from executable planning. Our approach integrates these concerns into A* search with domain-specific heuristics.

3. Problem Formulation

3.1. Graph Representation

We formulate the problem as a directed weighted graph $G = (V, E)$ where each node $v \in V$ represents a unique (exchange, stablecoin) pair and each directed edge $e \in E$ corresponds to a trade or a cross-exchange transfer. Every edge has an associated effective exchange rate incorporating all relevant costs, and we assign weight $w(e) = -\log(\mathit{effective\_rate}(e))$, converting multiplicative returns into additive costs for shortest-path algorithms.

3.2. Path Profitability

Given a path $P = (v_0, v_1, \ldots, v_k)$ with initial capital C_0, the final capital is $C_k = C_0 \cdot \prod_{e \in P} r(e)$ where $r(e)$ denotes the effective rate on edge e. A path is profitable when $C_k - C_0 \geq \mathit{min\_profit}$. In additive log space this becomes:

$$\sum_{e \in P} -\log\big(r(e)\big) \;\leq\; -\log\left(1 + \frac{\mathit{min\_profit}}{C_0}\right).$$

The goal set is $\mathcal{G} = \left\{ v_k \in V \;\middle|\; C_k > C_0 \right\}$. We do *not* require $v_k = v_0$: this **open-path** formulation allows the trader to end on any pair where USD value exceeds initial capital, justified because all assets are USD-pegged stablecoins ($\pm 2\%$ price tolerance).

3.3. A* Search Formulation

We apply A* search with cost-to-reach $g(n) = \sum_{e \in \text{path}} -\log(r(e))$ and evaluation function $f(n) = g(n) + h(n)$, where $h(n)$ estimates the execution risk of the remaining path given current blockchain state, including factors such as liquidity depth, order-book slippage, chain congestion, and exchange reliability. The algorithm terminates when $C_n > C_0$.

For h_4, we employ **Weighted A*[16]** with $f(n) = g(n) + w(n) \cdot h(n)$, where $w(n) = w_{\min} + (w_{\max} - w_{\min}) \cdot \rho(n)$ and $\rho(n) \in [0, 1]$ is a chain kickback risk score. When $w(n) = 1$ for all nodes, this reduces to standard A*. The algorithm is not complete: market data does not guarantee a profitable path from every starting node. Search terminates when a profitable path is found, the frontier is exhausted, or depth/time limits are reached.

4. Novel Heuristics for Arbitrage Search

We design three guidance heuristics (h_1, h_2, h_4) and one search-control strategy (h_3) to steer A* toward executable arbitrage routes. Each captures a different dimension of execution feasibility via an additive penalty $h(n) \geq 0$ in $f(n) = g(n) + h(n)$. These are *guidance penalties*, not admissible lower bounds.

4.1. h_1 – Liquidity Heuristic

The liquidity heuristic penalizes low-volume markets that cannot support the required trade size. We compute a liquidity ratio from 24-hour quote volume, the remaining arbitrage window T_{remain}, and order size:

$$\mathit{liquidity\_ratio} = \frac{(\mathit{24h\_vol}/86400) \cdot T_{remain}}{\mathit{order\_size\_usd}}, \quad h_1(n) = \lambda_{liq}\left(1 - \mathit{clamp}\left(\frac{\log_{10}(\mathit{liquidity\_ratio})+1}{2},\, 0,\, 1\right)\right).$$

This heuristic favours deep, actively traded markets and is time-aware.

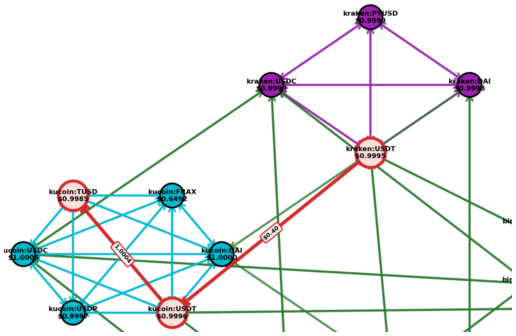


Figure 1. A profitable arbitrage path: kraken:USDT → kucoin:USDT → kucoin:TUSD. The first edge is a cross-exchange transfer ($r_1 = 0.9995$) and the second is an intra-exchange trade ($r_2 = 1.0042$), yielding a multiplicative return of ≈ 1.0037 ($37 profit on $10,000). This illustrates the open-path formulation: profitable paths need not form closed cycles.

4.2. h_2 – Slippage Heuristic

The slippage heuristic models execution price impact using real-time order-book depth. Given mid-price P_{mid} and the VWAP required to fill order size Q:

$$h_2(n) = \lambda_{slip} \cdot \max\{0,\ slippage_{bps} - \theta\}, \quad slippage_{bps} = \frac{VWAP(Q) - P_{mid}}{P_{mid}} \times 10^4,$$

where θ is a configurable tolerance. Edge costs encode static taker and withdrawal fees; h_2 captures *dynamic* order-book slippage, so there is no double-counting with $g(n)$.

4.3. Multi-Start Search Strategy (h_3)

h_3 runs k independent A* searches in parallel from randomly sampled starting nodes, each using a base heuristic, and returns the most profitable result.

4.4. h_4 – Chain Congestion and Exchange Reliability

h_4 penalizes transfer latency and operational risk using pre-computed chain transfer times and static reliability scores $s(n) \in [0, 1]$:

$$h_4(n) = \lambda_{chain}\left(\frac{t_{\min}}{T_{\text{remain}}}\right) + \lambda_{exchange}(1 - s(n))\,.$$

5. Experiments and Evaluation

5.1. Setup

We gather live market data from twelve major exchanges via CCXT[1], constructing graphs of up to 41 nodes and 864 edges. We evaluate via cached-graph heuristic comparison, graph scaling (4–12 exchanges), an 8-hour overnight campaign, and quote staleness.

5.2. Results

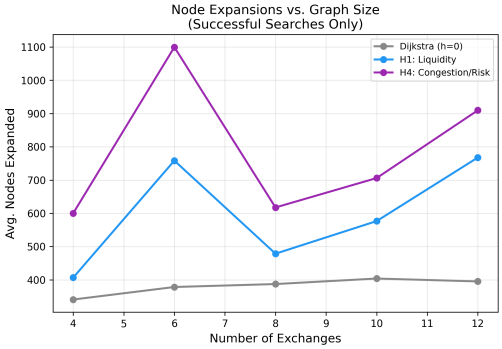
Tables 1 and 2 summarize heuristic performance. On the cached graph (30 start nodes), all A*-based methods achieve identical 56.7% success; Bellman–Ford, 1-hop, and 2-hop baselines fail entirely. h_2 reduces expansions by **29%** relative to Dijkstra while matching its profit within 1%, suggesting slippage-aware guidance follows the same optimal routes with

Table 1. Heuristic comparison at $10k (30 starts).

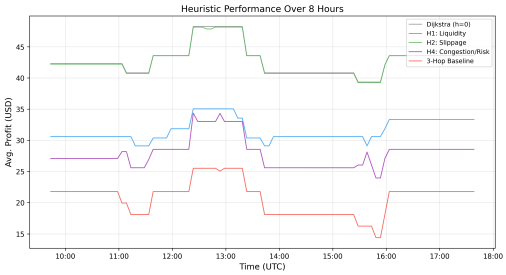
Heuristic	Succ.	Profit	Exp.	Δ
Dijkstra	56.7%	$10.01	359	—
h_1 (liquidity)	56.7%	$6.72	568	−58%
h_2 (slippage)	56.7%	$9.91	256	+29%
h_4 (Wt. A*)	56.7%	$7.06	673	−87%

Table 2. Node expansions by graph size (10 trials).

Ex.	$\|V\|$	$\|E\|$	Dijk.	$\mathbf{h_1}$	$\mathbf{h_4}$
4	19	161	341	407	600
6	28	368	378	758	1100
8	35	619	387	479	618
10	39	760	404	577	706
12	41	864	395	768	910



(a) Node expansions vs. graph size.



(b) Overnight comparison (1,440 runs each).

Figure 2. (a) Dijkstra expansions stay flat as graphs grow; penalty heuristics scale worse. (b) Profit, success, and expansions across the 8-hour campaign.

less wasted exploration. h_1 and h_4 expand 58–87% *more* nodes and find 30–33% less profit, indicating their risk penalties overcompensate and divert search from structurally optimal paths.

Over an 8-hour overnight campaign (**7,200 instances**), all A*-based methods achieve **100% success**; 3-hop enumeration succeeds in only 39.8%. h_2 matches Dijkstra-level profit across all order sizes while consistently expanding fewer nodes (Figure 2). Re-evaluating paths after 5–300 s delays, 99.6% remain profitable up to 120 s.

6. Conclusions

We presented an execution-aware A* framework for cross-exchange stablecoin arbitrage across 12 exchanges (41 nodes, 864 edges). h_2 (slippage) reduces expansions by 29% while matching Dijkstra's profit; h_1 and h_4 overcompensate risk, expanding more nodes for less profit. Future work includes asynchronous order-book pre-fetching and extension to DEX/AMM markets.

Acknowledgments

We thank Hang Ma and Tania Pocrnjic for helpful feedback on an earlier draft of this paper.

References

[1] CCXT. *CryptoCurrency eXchange Trading Library*. Accessed: 2024. 2024. URL: https://github.com/ccxt/ccxt.

[2] CoinMarketCap. *Stablecoin Market Capitalization*. Accessed: March 2025. 2025. URL: https://coinmarketcap.com/view/stablecoin/.
[3] The White House. *Executive Order: Establishing the Strategic Bitcoin Reserve and United States Digital Asset Stockpile*. March 6, 2025. 2025. URL: https://www.whitehouse.gov/presidential-actions/executive-order-establishing-the-strategic-bitcoin-reserve-and-united-states-digital-asset-stockpile/.
[4] R. Bellman. "On a routing problem". In: *Quarterly of Applied Mathematics* 16.1 (1958), pp. 87–90.
[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
[6] R. Oant and A. Coroiu. "Crypto Advisor: A Web Application for Spotting Cross-Exchange Cryptocurrency Arbitrage Opportunities". In: *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU 2023)*. Vol. 1. 2023, pp. 238–246. ISBN: 978-989-758-641-5. DOI: 10.5220/0011850400003470.
[7] L. Zhou, K. Qin, C. F. Torres, D. V. Le, and A. Gervais. "On the Just-In-Time Discovery of Profit-Generating Transactions in DeFi". In: *IEEE Symposium on Security and Privacy (Oakland)*. 2021.
[8] Anonymous. "An Improved Algorithm to Identify More Arbitrage Opportunities". In: *arXiv preprint arXiv:2406.16573* (2024).
[9] Anonymous. "Real-Time Identification of Negative Cycles for High-Efficiency Arbitrage". In: *VLDB* 18 (2025), pp. 4081–4094.
[10] F. Fang, C. Ventre, M. Basios, L. Kanthan, D. Martinez-Rego, F. Wu, and L. Li. "Cryptocurrency trading: a comprehensive survey". In: *Financial Innovation* 8.1 (2022), pp. 1–59.
[11] A. Obadia, A. Salles, L. Sanchez, T. Sri, V. Buterin, and P. Daian. "Unity is Strength: A Formalization of Cross-Domain Maximal Extractable Value". In: *arXiv preprint arXiv:2112.01472* (2021).
[12] Anonymous. "Optimal Execution with Reinforcement Learning". In: *arXiv preprint arXiv:2411.06389* (2024).
[13] E. Nikolova and D. R. Karger. "Route Planning under Uncertainty: The Canadian Traveller Problem Revisited". In: *Proceedings of the 23rd National Conference on Artificial Intelligence (AAAI)*. 2008, pp. 969–974.
[14] S.-H. Lim, I.-J. Rhee, and M. C. Fu. "Risk-averse shortest path problems". In: *Mathematics of Operations Research* 47.3 (2022), pp. 2001–2029.
[15] R. Almgren and N. Chriss. "Optimal execution of portfolio transactions". In: *Journal of Risk* 3.2 (2001), pp. 5–39.
[16] I. Pohl. "Heuristic search viewed as path finding in a graph". In: *Artificial Intelligence* 1.3-4 (1970), pp. 193–204.

Appendix A. Appendix

A.1. Hyperparameter Configuration

Table 3 lists all hyperparameters used in the heuristic functions and search algorithms. Values are fixed across all experiments.
Chain transfer times. Transfer-time estimates are drawn from documented blockchain finality times plus typical exchange processing delays: Solana $\approx 1\,\text{s}$, BNB Chain $\approx 4\,\text{s}$, Polygon $\approx 5\,\text{s}$, Tron $\approx 30\,\text{s}$, Arbitrum/Base $\approx 120\,\text{s}$, Ethereum L1 $\approx 600\,\text{s}$. These are used by h_4 to compute chain reliability penalties and by the transfer-time model to assess path time-feasibility.

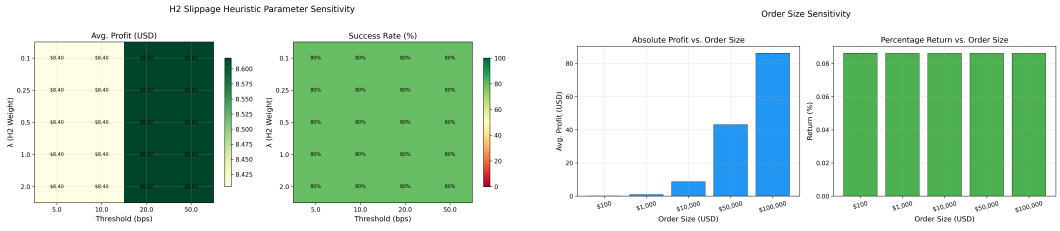
A.2. Sensitivity Analysis

h_2 slippage parameters. A joint sweep of $(\lambda_{\text{slip}}, \theta)$ across 100 configurations shows a broad plateau of high performance at moderate values ($\lambda_{\text{slip}} \in [0.3, 1.0]$, $\theta \in [5, 20]$ bps), confirming h_2 is not sensitive to precise tuning (Figure 3a).

Table 3. Hyperparameters and their values.

Parameter	Description	Value
H1 – Liquidity Heuristic		
λ_{liq}	Liquidity penalty weight	1.0
Unknown penalty	No-data fallback cost	5.0
Cache TTL	Volume cache lifetime	60 s
H2 – Slippage Heuristic		
λ_{slip}	Slippage penalty weight	0.5
θ	Slippage tolerance	10 bps
Unknown penalty	No-data fallback cost	50.0
Cache TTL	Order-book cache lifetime	30 s
H4 – Chain/Exchange Risk		
λ_{chain}	Chain risk penalty weight	1.0
$\lambda_{\text{exchange}}$	Exchange risk penalty weight	1.0
$w_{\min}$	Min Weighted A* weight	1.0
$w_{\max}$	Max Weighted A* weight	3.0
Search Parameters		
Max depth	Maximum path hops	6
$T_{\max}$	Arbitrage time window	1800 s
Early exit iter.	Iters after first profit	200
k (H3)	Multi-start parallel runs	3
User overhead	Execution latency per hop	45 s

Order size. Profit scales linearly from \$0.09 at \$100 to \$86.18 at \$100k, with constant 80% success rate (Figure 3b).



(a) h_2: success rate across $(\lambda_{\text{slip}}, \theta)$ grid.

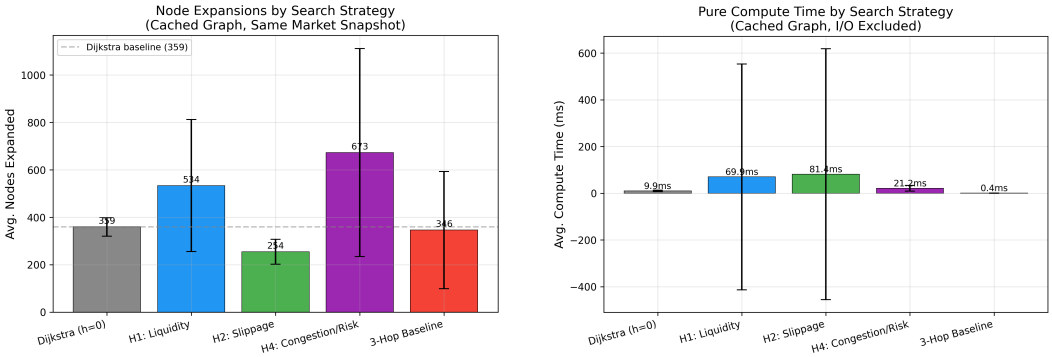
(b) Profit vs. order size (\$100–\$100k).

Figure 3. (a) h_2 is robust across a wide parameter range. (b) Profit scales linearly with order size; success rate is constant.

A.3. **Additional Experimental Results**

Heuristic comparison: expansions and compute time. Figure 4 compares mean node expansions and compute time across all heuristics on the cached graph. h_2 consistently achieves the fewest expansions, while h_4 matches Dijkstra-class latency due to pre-computed scores requiring zero API calls.

Quote staleness. Re-evaluating discovered paths against fresh market data after delays of 5–300 s (270 measurements), **99.6%** remain profitable up to 120 s; only at 300 s does a single path lose profitability (98% survival), demonstrating strong structural persistence of stablecoin arbitrage paths.



(a) Mean node expansions by heuristic.

(b) Mean compute time by heuristic.

Figure 4. Heuristic comparison on cached graph (30 start nodes, 3 cash levels).

Table 4. Quote staleness: path survival and profit change.

Delay (s)	**Paths**	**Profitable**	**Mean Decay (%)**
5	45	100%	−64.6
10	45	100%	−67.0
30	45	100%	−71.6
60	45	100%	−72.7
120	45	100%	−72.4
300	45	98%	−55.0